

Programming Languages

Departamento de Engenharia Informática, Técnico Lisboa

MEIC P4 25.26

Project Statement (Phase 2 Rusty 2)

Version 1 (18-May-2026)

© Luís Caires (luis.caires@tecnico.ulisboa.pt | https://luiscaires.org)

Objectives of PL 25-26 Phase 2 Project

Implement an interpreter for the Rusty2 functional-imperative language

- The starting point for Phase 2 is your interpreter for Rusty 1. You are expected to:
  - Base your implementation in the big-step environment-based semantics studied in the course, extended with the following features.

- **Baseline** Grading (graded for up to 17)
  - Static Type-checking and (non-recursive) type definitions for all of Rusty1.
  - Labeled Sum (option) types with a (special kind of) width and depth **subtyping.**

- **Points of Excellence** Grading (from 18 to 21!)
  - Recursive Type Definitons and Static Type checking for Recursive types (graded 1)
  - Type Safe Stack Allocated Mutable Cells (**let mut**) (graded 1)
  - Type Safe Aliases (**&x**) (graded **2**, so includes a bonus that will add to the **course final grade**)

Rusty2 base language for Project Phase 2

```
EE -> id | num | true | false | string | ()
    | EE + EE | EE - EE
    | EE * EE | EE / EE | -EE |  ( EE )
    | EE == EE | EE > EE | EE >= EE | …
    | EE && EE |  EE || EE |  ~ EE
    | tag(EE,EE,…)
    | match EE { tag(id,id,…) => EE, … }
    | let id (: A)? = EE)+ ; EE
    | type id = A ; EE
    | cell EE | EE := EE | ! EE
    | if EE then EE else  EE end
    | while EE do EE end
    | print EE | | println EE | EE ; EE
    | fn ( id, A, EE )
    | app ( EE, EE)
```

```
A,B -> id
     | ()
     | int
     | bool
     | string
     | A -> B
     | enum { tag: ( A,B,C, … ), … }
     | ref ( A )
```

Note: This is almost a statically typed version of your Phase 1 language. new features / changes under yellow

Rusty 2 base language for Project Phase 2

- **Type Expressions** % see grammar fragment in the drive folder
  - TE ::= TF (**->** TE) ? % right associative
  - TF ::= **unit** | **int** | **bool** | **string** | **ref** (T) |
    enum { (**tag :** T)* } | **(** TE **)** | **id**

  - **You will need to create ASTType nodes to represent the AST of type expressions.**

- **Scoped Type Definitions** % see grammar fragment in the drive folder
  - Let ::= Seq | (**let id** = T;)+Seq | (**type id** = T;)+ Seq

Rusty 2 - basic notes on semantics

- The semantics of Rusty 2 conforms and extends the big step semantics covered in the lectures, and most of the typing rules as well.
- The **match** construct will now just apply to union types (called here **enum**).
- The struct tuple of Phase 1 will be represented in Rust 2 by a union type with a single option, also “eliminated” using match.
- This "new” **match** construct does not require an **else** branch, since Rusty 2 is statically typed.

Static Type Checking (reminder)

Typing Rules (Basic Functional Core)

$$\frac{\Gamma(x) = A}{\Gamma \vdash x : A} \qquad \Gamma \vdash n : \text{int} \qquad \Gamma \vdash b : \text{bool}$$

$$\frac{\Gamma \vdash M : \text{int} \quad \Gamma \vdash N : \text{int}}{\Gamma \vdash M + N : \text{int}} \qquad \frac{\Gamma \vdash M : \text{int} \quad \Gamma \vdash N : \text{int}}{\Gamma \vdash M \leq N : \text{bool}} \qquad \frac{\Gamma \vdash M : \text{bool} \quad \Gamma \vdash N : \text{bool}}{\Gamma \vdash M \;\&\&\; N : \text{bool}}$$

$$\frac{\Gamma \vdash N : C \quad C <: A \quad \Gamma \vdash M : A \to B}{\Gamma \vdash MN : B} \qquad \frac{\Gamma, x{:}A \vdash M : B}{\Gamma \vdash \lambda x{:}A.M : A \to B}$$

$$\frac{\Gamma \vdash M : \text{bool} \quad \Gamma \vdash M : A \quad \Gamma \quad \Gamma \vdash R : A}{\Gamma \vdash \text{if}(M, N, R) : A} \qquad \frac{\Gamma \vdash N : B \quad \Gamma, x : B \vdash M : A}{\Gamma \vdash \text{let } x = N \text{ in } M : A}$$

Static Type Checking (reminder)

Typing Rules (References)

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \mathsf{cell}(M) : \mathsf{ref}(A)}$$

$$\frac{\Gamma \vdash M : \mathsf{ref}(A)}{\Gamma \vdash\ !M : A}$$

$$\frac{\Gamma \vdash M : \mathsf{ref}(A) \quad \Gamma \vdash N : A}{\Gamma \vdash M := N : A}$$

Static Type Checking (reminder)

Typing Rules (Products and Sums)

$$\frac{\Gamma \vdash M : A_i}{\Gamma \vdash l_i(M) : \mathsf{enum}\ \{\ l_1 : A_1, \ldots, l_n : A_n\ \}}$$

$$\frac{\Delta \vdash M : \mathsf{union}\ \{\ l_1 : A_1, \ldots, l_n : A_n\ \} \quad \Delta, x{:}A_i \vdash N_i : C}{\Delta \vdash \mathsf{match}\ M\ \mathsf{of}\ \{\ l_i(x) \to N_i\ \} : C}$$

Static Type Checking

Basic SubTyping Rules

$$\frac{}{A <: A} \qquad \frac{A <: B \quad B <: C}{A <: C} \qquad \frac{A <:> B}{\mathsf{ref}(A) <: \mathsf{ref}(B)} \qquad \frac{C <: A \quad B <: D}{A \to B <: C \to D}$$

$$\frac{(\text{all } i \in 1...k)\ A_i = (C_1, \ldots, C_{a_i}, \ldots, C_{b_i}) <: B_i = (D_1, \ldots, D_{a_i}) \quad C_p <: D_p \quad (\text{all } p \in p \ldots a_i)}{\mathsf{enum}\ \{l_1 : A_1, \ldots, l_k : A_k\} <: \mathsf{enum}\ \{l_1 : B_1, \ldots, l_k : B_k, \ldots, l_n : B_n\}}$$

Subtyping (examples)

Rusty 1 enum types combine both with and depth subtyping. Examples:

- **enum** { #person:(**string**, **int**) } <: **enum** { #person:(string) }
  This corresponds to record subtyping, a subtype can have “more” fields (at the end).

- **enum** { #pizza:(**string**) } <: **enum** { #pasta:(**string**), #pizza:(**string**) }
  This corresponds to union subtyping, a subtype may have “less” options.

- **enum** { #pizza:(**string, int**) } <: **enum** { #pasta:(**int**), #pizza:(**string**) }
  Both modes can be combined!

Challenge 1 (recursive types)

Challenge 1 (recursive types)

- The Rusty 1 language may support recursive types like in the following example.

```
type Btree = enum { #nil: (unit), #node: (Btree, int, Btree) } ;
let countNodes =
     |t:Btree | {
        match t {
               Nil(_) -> 0
          |    Node(l,v,r) -> 1 + (countNodes (l)) + (countNodes (r))
        }
     };  … ;;
```

- More examples and tests will be released soon in the Course Drive.

Challenge 2 (stack allocated mutable variables)

Challenge 2 (stack allocated mutable variables)

- The Rusty 1 language supports heap allocated cells (**cell**).
- But we would like to also support stack allocated mutable cells. We illustrate with a simple example

```
let main = |l:int | {
  let mut a = l;
  let mut s = 0;
  while (a>=0) {
      s := s + a;
      a := a - 1
  };
  s
};
println!(main(10))
```

As discussed in the lectures the lifetime of **mut** variables is exactly the scope in which they are declared.

Challenge 2 (stack allocated mutable variables)

- The Rusty 1 language supports heap allocated cells (**cell**).
- But we would like to also support stack allocated mutable cells. We illustrate with a simple example

```
let main = |l:int | {
  let mut a = l;
  let mut s = 0;
  while (a>=0) {
      s := s + a;
      a := a - 1
  };
  s
};
println!(main(10))
```

As runtime, **mut** variables must be **allocated** at declaration time and **deallocated** at the end of scope, in a stack LIFO discipline. I provide a schematic memory manager for you to use in the **shared drive**.

Challenge 2 (stack allocated mutable variables)

The provided memory manager uses a specific value type **VAddress** to denote references to stack allocated cells, and provides the following interface.

```
public class MemoryManager {

    public class VAddress { }

    MemoryManager(int size) { }

    MemoryManager()  {}

    VAddress push(IValue v) throws MemoryPanicError { }

    VAddress pop() throws MemoryPanicError { }

    IValue memrd(VAddress address) throws MemoryPanicError{ }

    void memwrt(VAddress address, IValue v) throws MemoryPanicError{ }

}
```

Challenge 2 (stack allocated mutable variables)

- During program execution, **mut** variables must be **allocated** at the point of declaration and **deallocated** at the end of scope, in a stack LIFO discipline.

```
let main = |l:int | {
  let mut a = l;
  let mut s = 0;
  while (a>=0) {
      s := s + a;
      a := a - 1
  };
  s
  };
println!(main(10))
```

mem.push(..)

mem.push(..)

mem.pop() ; mem.pop()

Challenge 2 (stack allocated mutable variables)

- Stack-allocated variables are handled in the **L-value/R-value** mode usual in usual imperative programming languages, **implicitly dereferenced** on reads:

```
let main = |l:int | {
  let mut a = l;
  let mut s = 0;
  while (a>=0) {
      s := s + a;
      a := a - 1
  };
  s
};
println!(main(10))
```

s in an expression denotes the contents of a

s on the left of (:=) denotes the reference to the cell to be updated

s in an expression denotes the contents of a

Challenge 2 (stack allocated mutable variables)

- What the “same” program would be with heap-allocated instead. The overall effect of the two programs is similar, but the using **mut** cells does not create garbage, while using **cell** does.

```
let main = |l:int | -> int {
  let a = cell(l);
  let s = cell(0);
  while (!a>=0) {
      s := !s + !a;
      a := !a - 1
  };
  s
};
println!(main(10))
```

- The implicit L/R usage of **mut** variables yields cleaner code, dispensing explicit dereference (!)

Challenge 2 (stack allocated mutable variables)

- Inspired by the language **R1r** described in the lectures, implement the concrete syntax, abstract syntax, evaluation and type-checking to support **mut** declarations in Rusty 2.
- Your type system **must ensure** that **no memory leaks** or **illegal dereferences** will be possible in any well-typed Rusty 2 programs. Notice that the memory manager will **panic** if than happens.
- To achieve this, **your type system** must ensure that no usages (reads or writes) of any **mut** variable will be made outside its life time.
- You will need to introduce a **new type mut**(A) to annotate identifiers as **mut** variables during type-checking.
- You also need to add new typing rules and perhaps modify some of the already existing ones.

Challenge 2 (stack allocated mutable variables)

- Type **mut**(A) will be **JUST** for internal use of the type-checker, that is, the **mut** keyword will never appear in the syntax of types, only in **let mut** declarations, as suggested by the following rules:

$$\Gamma, x : \mathsf{mut}(A) \vdash x : A$$

$$\frac{\Gamma \vdash N : B \quad \Gamma, x : \mathsf{mut}(B) \vdash M : A}{\Gamma \vdash \mathsf{let\ mut}\ x = N\ \mathsf{in}\ M : A}$$

- At runtime, e.g., in the interpreter, identifiers of **mut**(A) type will be always bound to **VAddress** values (in the interpreter).
- Notice that Rusty 2 will still provide heap-allocated cells, represented by values of **ref**(A) type, **not** of values **VAddress** of **mut**(A) type.

Challenge 2 (stack allocated mutable variables)

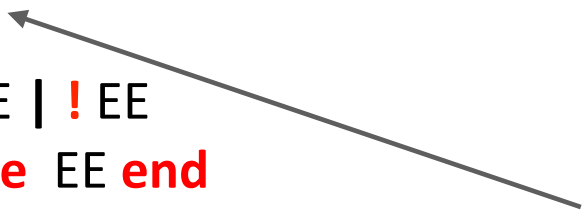
- Example of a well-typed program well-typed using both with heap and stack allocated cells:

```
let global = cell(10);
let side =
   |l:ref(int) | -> () {
  let mut a = !l;
  let mut s = 0
  while (a>=0) {
      s := s + a;
      a := a - 1
  };
  l := s
};
side(global);
println!(global)
```

Rusty2 language for Project Phase 2 (challenge 2)

```
EE -> id | num | true | false | string | ()
    | EE + EE | EE - EE
    | EE * EE | EE / EE | -EE |  ( EE )
    | EE == EE | EE > EE | EE >= EE | …
    | EE && EE |  EE || EE |  ~ EE
    | tag(EE,EE,…)
    | match EE { tag(id,id,…) => EE, … }
    | let id ( (mut)? (: A)? = EE)+ ; EE
    | type id = A ; EE
    | cell EE | EE := EE | ! EE
    | if EE then EE else  EE end
    | while EE do EE end
    | print EE | | println EE | EE ; EE
    | fn ( id, A, EE )
    | app ( EE, EE)
```

```
A,B ->  id
      | ()
      | int
      | bool
      | string
      | A -> B
      | enum { tag: ( A,B,C, … ), … }
      | ref ( A )
```



(**Bonus**) Challenge 3 (explicit aliasing)

(**Bonus**) Challenge 3 (explicit aliasing)

- Your Rusty 2 language now supports heap allocated cells (**cell**) and stack allocated cells (**mut**)
- But Challenge 3 will provide support for aliases to mut allocated cells. Look the following example.

```
let mut s = 1;
let p = &s;
p := !p + s;
println!(s)
```

- The expression &s can only be used when s is a mut variable name in the scope.
- Taking aliases returns a reference value of type **addr**(A), which is conceptually different from a reference of reference type **ref**(A). In particular, like for cell references, a value of type **addr**(A), may be dereferenced with !M, as in the example above.
- Unlike **mut**(A), the type **addr**(A) will be present at the type language, and may be used by the programmer. Values of type **addr**(A) will be represented by **VAddress** objects.

(**Bonus**) Challenge 3 (explicit aliasing)

- In this challenge you will extend your Rusty 2 language type-checker and interpreter to handle explicit aliases, and design and implement typing rules that will **ensure** that **no memory leaks** or **illegal dereferences** will be possible in any well-typed programs!
- For example, the following code will be rejected the type system.

```
let flop = |n:int| -> &int
     {  mut s = n;
        let p = &s;
        p := !p + s;
        p
    };
 println!(flop(2))
```

(**Bonus**) Challenge 3 (explicit aliasing)

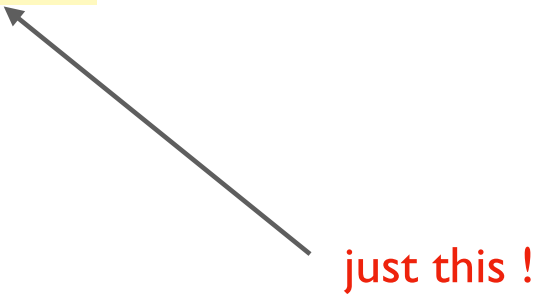
- In this challenge you will extend your Rusty 2 language type-checker and interpreter to handle explicit aliases, and design and implement typing rules that will **ensure** that **no memory leaks** or **illegal dereferences** will be possible in any well-typed programs!
- But , the following code will type-checkl (prints 3)!

```
let inc = |p:&int| -> ()
     {
       p := !p + 1; ()
      };
let mut a = 2;
inc (&a);
println!(a)
```

Rusty2 language for Project Phase 2 (challenge 3)

```
EE -> id | num | true | false | string | ()
    | EE + EE | EE - EE
    | EE * EE | EE / EE | -EE |  ( EE )
    | EE == EE | EE > EE | EE >= EE | …
    | EE && EE |  EE || EE |  ~ EE
    | tag(EE,EE,…)
    | match EE { tag(id,id,…) => EE, … }
    | let id ( (mut)? (: A)? = EE)+ ; EE
    | type id = A ; EE
    | cell EE | EE := EE | ! EE
    | if EE then EE else  EE end
    | while EE do EE end
    | print EE | | println EE | EE ; EE
    | fn ( id, A, EE )
    | app ( EE, EE)
```

```
A,B ->  id
      | ()
      | int
      | bool
      | string
      | A -> B
      | enum { tag: ( A,B,C, … ), … }
      | ref ( A )
      | & ( A )
```



Some Practical Features

Some Practical Features

1 - the executable should be executable from a sh script named “rs2” as in

```
luis@macbook ~ > rs2
# 2;;
2
^D
luis@macbook ~
```

2 - We should also be able to run code from a file indicated in the command line as in

```
luis@macbook ~ rs2  hello.rs2
1 2 3 4 5 6 7 8 9 10
luis@macbook ~
```

Some Practical Features

3 - you should include a flag -types that deactivates the type-checker.

```
luis@macbook ~ > rs2 -types
# 2+true;;
InterpreterError: illegal types to + operator
   at ASTPlus.eval(ASTPlus.java:16)
   at R0int.main(R0int.java:15)
```

Submission Instructions

1 - Submit all code to the gitlab / github repo shared with me (luis.caires@tecnico.ulisboa.pt). We may store everything in a folder well-named.

2 - Include a small report (pdf) briefly explaining how you have implemented the static type checker and possibly the recursive types extension.

NOTE: the contents of the shared repo **must not be changed** after the deadline.

3- You may use the software available in the gitlab RNL standard installation and javacc (installation requested).

4 - Include a top level script “makeit”, that I may use to compile all your source code by typing “\.makeit” at the command line.

5- Include some rs2 examples to demonstrate your interpreter. You must include original new examples of your own. If you have

Use the Slack to clarify any aspects